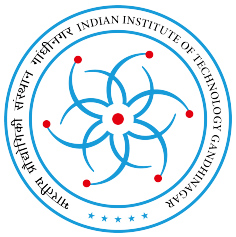

Indian Institute of Technology

Department of Computer Science

CS202: Software Tools and Technologies
Assessment 2

Jaskirat Singh Maskeen and Aarsh Wankar

(23110146, 23110003)

Instructor: Prof. Shouvick Mondal

September 30, 2025

Contents

Lab 1

Evaluation of Vulnerability Analysis Tools using CWE-based Comparison

Github Repository for Lab 6

1.1 Introduction

In this lab we evaluate and compare different static application security testing (SAST) tools based on their ability to detect software weaknesses in real-world, large-scale open-source projects. This comparison is based on the Common Weakness Enumeration (CWE), a community-developed list of the Top 25 most dangerous software and hardware weaknesses. By analyzing the CWEs reported by each tool, we can see their respective strengths and weaknesses, as well as their overall coverage of the top 25 CWEs. Furthermore, we will use metrics such as Intersection over Union (IoU) to quantify the pairwise agreement and disagreement between the tools, helping us understand their similarity and diversity in vulnerability detection.

1.2 Setup and Tools

We have Git, Python, and VSCode installed. `git --version` gives `git version 2.45.2.windows.1` as the output. The operating system on which all commands and analysis are performed is Windows 11. SEART Search Engine `https://seart-ghs.si.usi.ch/` is used to search and filter the repository based on our criteria. The working directory is named `STT_Lab_6`.

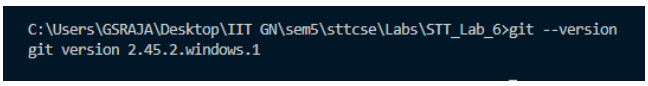

Figure 1.1: Git Version

1.3 Methodology and Execution

We used the tool SEART Search Engine to search for repositories. The selection criteria for these repositories is as follows Fig. 1.2:

- Language should be Java.
- There should be a minimum of 2 branches and 10 releases. (So that we can see the evolution of the code).
- The repository should have at least 10000 stars, indicating its popularity and activeness, and atmost 30000 stars, to avoid extremely large repositories.

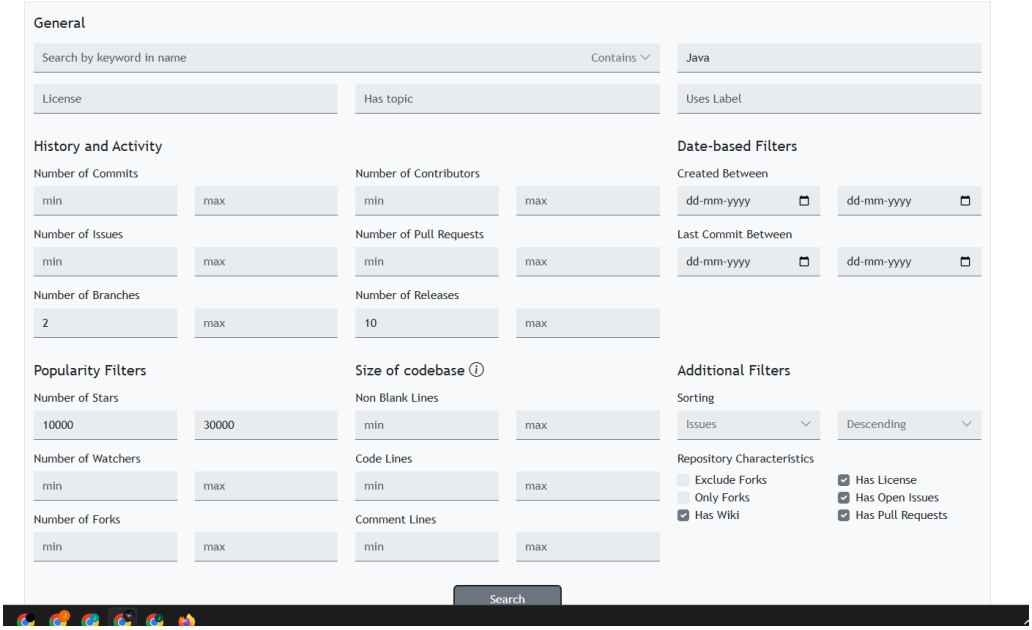

Figure 1.2: Search Criteria

- The repositories should have a Wiki, License, Open Issues, and Pull Requests.

Based on the results, we selected three large-scale, popular open-source repositories: **dagger**, **guice**, and **sentinel**.

The following three static analysis tools, which explicitly support CWE-based vulnerability reporting, were chosen for the evaluation:

- CodeQL: A powerful semantic code analysis engine.
- Semgrep: A fast, open-source static analysis tool for finding bugs and enforcing code standards.
- Snyk: A developer security platform that helps to find and fix vulnerabilities in code, open source dependencies, containers, and infrastructure as code.

Each of the three selected tools were run on all three chosen projects. The results of each scan were exported in the SARIF (Static Analysis Results Interchange Format), which is a structured JSON format. These SARIF files are available in the GitHub repository for this lab.

```
C:\Users\GSRAJA\Desktop\IIT GN\sem5\sttcse\Labs\STT_Lab_5>git clone https://github.com/alibaba/sentinel
Cloning into 'sentinel'...
remote: Enumerating objects: 27926, done.
remote: Counting objects: 100% (32/32), done.
remote: Compressing objects: 100% (25/25), done.
remote: Total 27926 (delta 23), reused 7 (delta 7), pack-reused 27894 (from 3)
Receiving objects: 100% (27926/27926), 8.18 MiB | 10.89 MiB/s, done.
Resolving deltas: 100% (10877/10877), done.
Updating files: 100% (1605/1605), done.

C:\Users\GSRAJA\Desktop\IIT GN\sem5\sttcse\Labs\STT_Lab_5>git clone https://github.com/google/guice
Cloning into 'guice'...
remote: Enumerating objects: 236640, done.
remote: Total 236640 (delta 0), reused 0 (delta 0), pack-reused 236640 (from 1)
Receiving objects: 100% (236640/236640), 122.37 MiB | 16.88 MiB/s, done.
Resolving deltas: 100% (199199/199199), done.

C:\Users\GSRAJA\Desktop\IIT GN\sem5\sttcse\Labs\STT_Lab_5>git clone https://github.com/google/dagger
Cloning into 'dagger'...
remote: Enumerating objects: 108403, done.
remote: Counting objects: 100% (145/145), done.
remote: Compressing objects: 100% (77/77), done.
remote: Total 108403 (delta 79), reused 75 (delta 62), pack-reused 108258 (from 4)
Receiving objects: 100% (108403/108403), 168.72 MiB | 13.83 MiB/s, done.
Resolving deltas: 100% (74025/74025), done.
Updating files: 100% (3419/3419), done.
```

Figure 1.3: Cloning the chosen repositories.

The setup for semgrep involved installing It via pip and setting an API token (from the website) as an environment variable followed by logging in via the command line (Fig. 1.4). For snyk, we installed it via

npm, and then set up authentication using CLI (Fig. 1.5). CodeQL was set up by downloading the CLI from the official website and configuring it in the PATH.

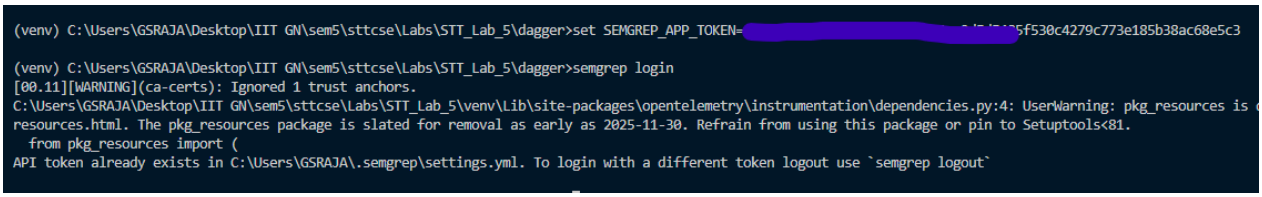

Figure 1.4: Semgrep Setup

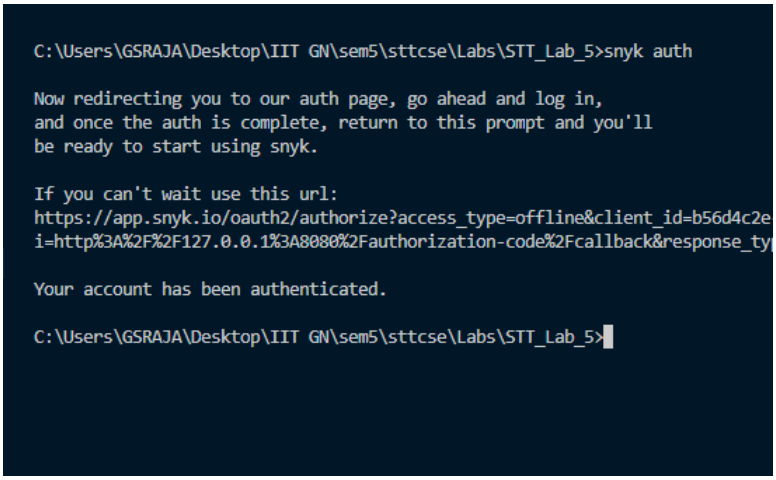

Figure 1.5: Snyk Setup

We then had to build some of the repositories, for example, dagger and guice, using `./gradle build` command (Fig. 1.6).

Figure 1.6: Gardle Build for dagger

After building, we ran the tools on the repositories. For all three repositories (example here, dagger), the commands used were as follows:

- **CodeQL**: `codeql database create ../dagger-db -language=java-kotlin -build-mode=autobuild`
  followed by
  `codeql database analyze ../dagger-db codeql/java-queries:codeql-suites/java-security-and-quality.qls -format=sarif-latest -output=codeql_dagger-results.sarif`
- **Semgrep** (Fig. 1.7): `semgrep -config=auto ../dagger -json -output=semgrep_results_dagger.sarif`

- **Snyk** (Fig. 1.8): `snyk code test -sarif-file-output=../snyk_results_dagger.sarif`

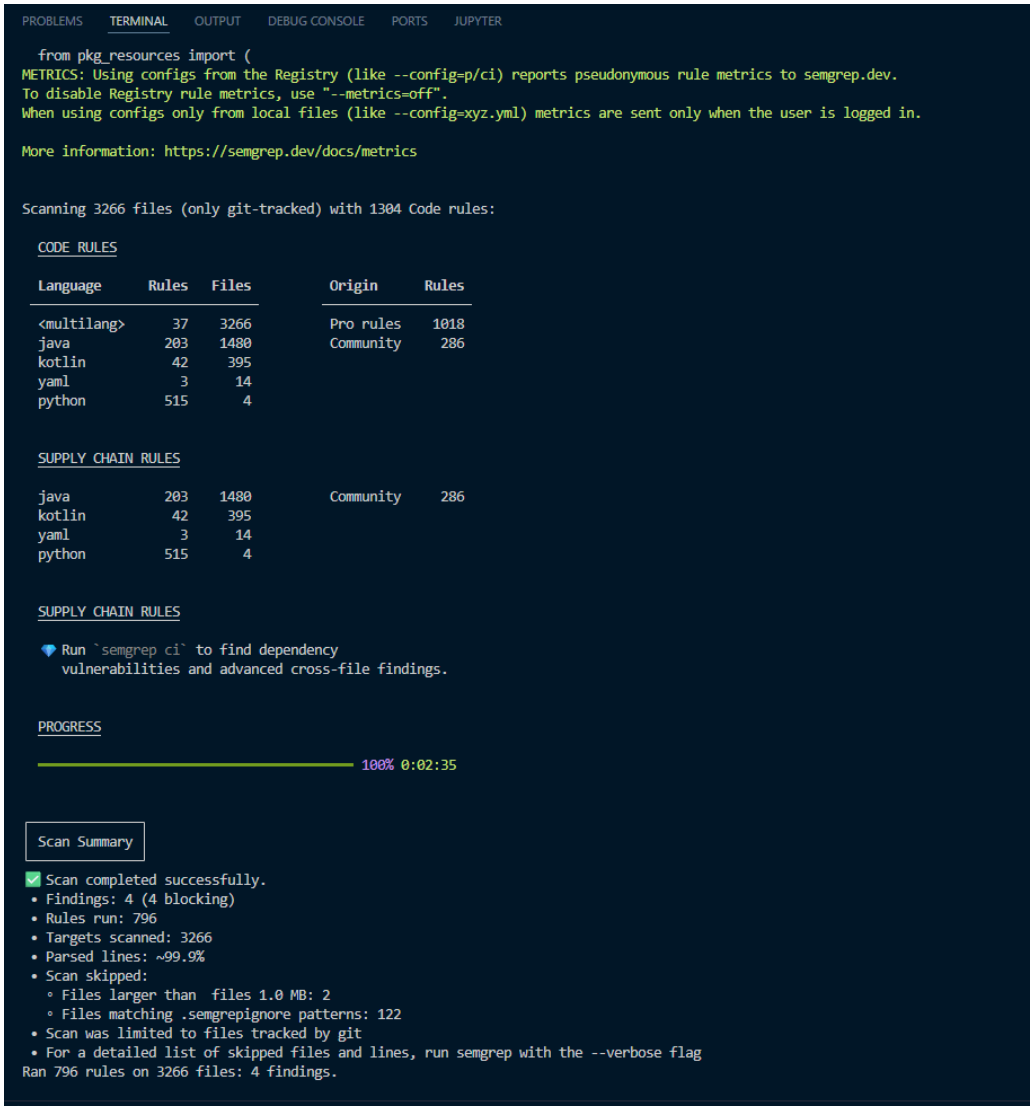

Figure 1.7: Semgrep Run for dagger

Note that CodeQL run for dagger had a very long output, so the STDOUT was stored as a file, available on the github repository.

We also write a ipynb notebook to parse the SARIF files and extract the CWE IDs reported by each tool for each repository. The findings were aggregated for each project-tool combination, counting the number of occurrences for each detected CWE ID. This data was consolidated into a single CSV file with the columns: **Project_name**, **Tool_name**, **CWE_ID**, and **Num_findings**. Additionally, each CWE ID was checked against the 2024 CWE Top 25 list, and a boolean column **Is_in_top_25_CWE** was added to the csv to indicate whether the finding belongs to this critical set of vulnerabilities.

We also store a csv file to show the overall coverage of each tool across all three projects, indicating the total number of unique CWE IDs detected by each tool againt all top 25 CWEs.

Finally, we calculated the Intersection over Union (IoU) metric for each pair of tools across all three projects. The IoU was computed as the size of the intersection of the sets of CWE IDs detected by each tool divided by the size of the union of these sets. This metric tells us about the agreement and disagreement between the tools in terms of the vulnerabilities they detect. We plot these as venn diagrams for visual clarity.

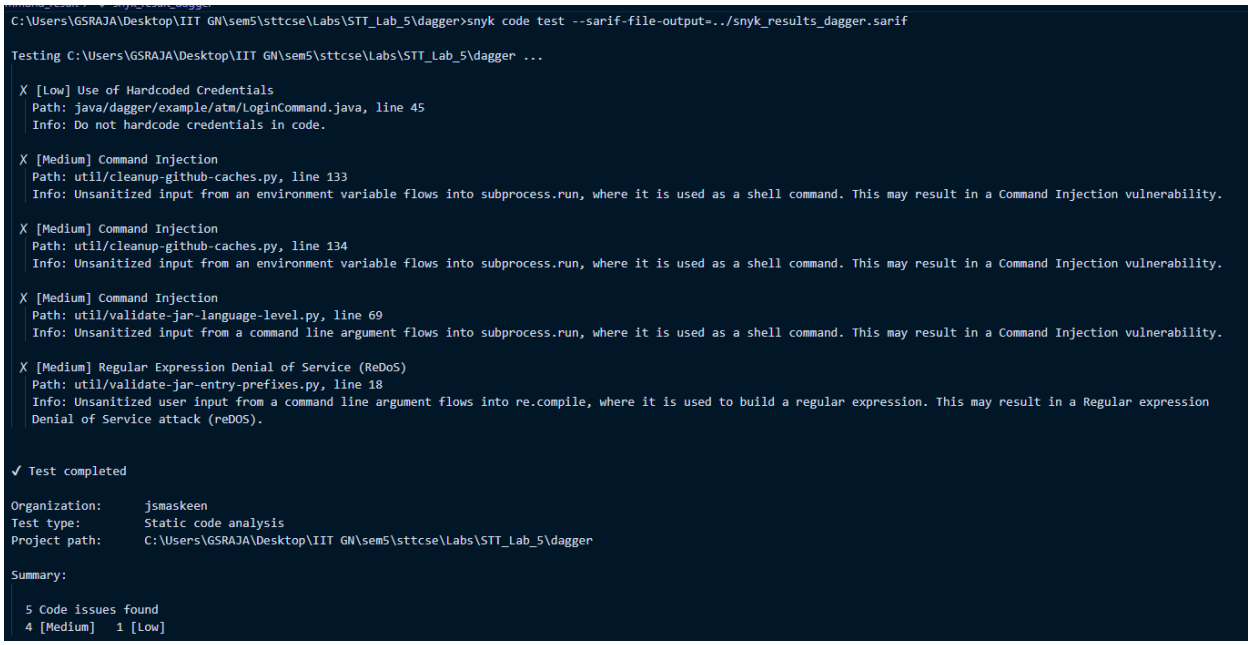

Figure 1.8: Snyk Run for dagger

1.4 Results and Analysis

We obtain the top 25 CWEs from https://cwe.mitre.org/data/csv/1430.csv.zip.

1.4.1 Pairwise findings

A sample of pairwise findings for the three tools across all three projects is shown in the table below (Table 1.1). The complete findings are available in the GitHub repository.

Project	Tool	CWE ID	Num Findings	In Top 25 CWE
dagger	CodeQL	cwe-248	2	False
dagger	CodeQL	cwe-312	14	False
dagger	Semgrep	cwe-78	3	True
dagger	Semgrep	cwe-798	1	True
dagger	Snyk	cwe-400	2	True
dagger	Snyk	cwe-78	6	True
guice	CodeQL	cwe-476	10	True
guice	CodeQL	cwe-477	4	False
guice	Semgrep	cwe-489	2	False
guice	Snyk	cwe-1004	2	False
guice	Snyk	cwe-23	2	False
sentinel	CodeQL	cwe-079	10	True
sentinel	CodeQL	cwe-117	74	False
sentinel	Semgrep	cwe-23	4	False
sentinel	Semgrep	cwe-319	2	False
sentinel	Snyk	cwe-208	2	False
sentinel	Snyk	cwe-23	14	False

Table 1.1: Sample findings (2 rows per project-tool pair).

1.4.2 Tool level CWE coverage

Table 1.2 shows the overall coverage of each tool across all three projects, indicating the total number of unique CWE IDs detected by each tool against all top 25 CWEs. The CWE IDs are truncated for readability.

Tool	CWE IDs Detected (truncated)	Top 25 CWEs Covered	Coverage (%)	#Top 25 CWEs
CodeQL	{cwe-571, cwe-209, ...}	5	20.0	5
Semgrep	{cwe-79, cwe-319, ...}	3	12.0	3
Snyk	{cwe-79, cwe-1004, ...}	6	24.0	6

Table 1.2: Coverage of Top 25 CWEs by different tools (CWE list truncated for readability).

As shown in the bar chart below (Fig. 1.9), Snyk has the highest coverage of the CWE Top 25 at 24%, followed by CodeQL at 20%, and Semgrep at 12%.

To visualize the specific CWEs detected by each tool, a heatmap was generated (Fig. 1.10), showing the coverage of each tool against the CWE Top 25 list.

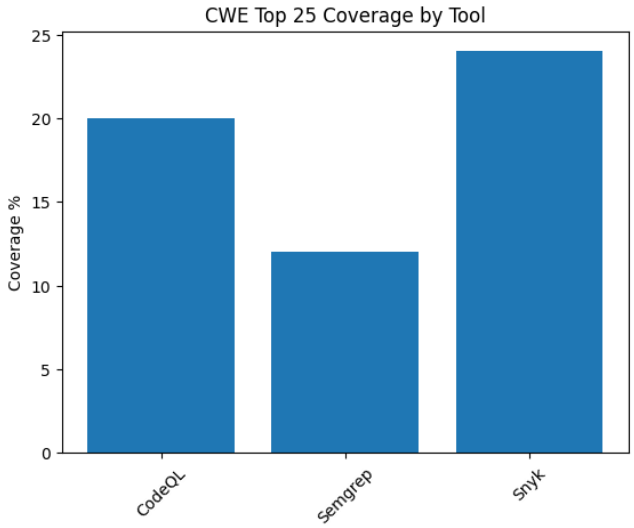

Figure 1.9: Bar chart showing coverage of Top 25 CWEs by different tools.

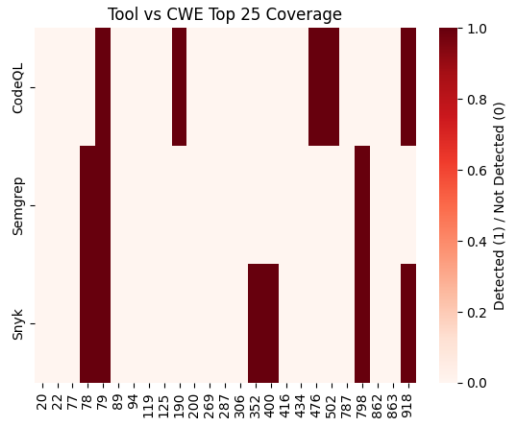

Figure 1.10: Heatmap showing coverage of each tool against the CWE Top 25 list.

1.4.3 Pairwise IoU Analysis

To understand the similarity and diversity of the tools, we computed the Intersection over Union (IoU), also known as the Jaccard Index, for each pair of tools on a per-project basis. The IoU value ranges from

0 to 1, where 1 indicates perfect agreement (the tools find the exact same set of CWEs) and 0 indicates no agreement (the tools find completely different sets of CWEs).

Dagger project

The IoU values for the dagger project are as follows:

	CodeQL	Semgrep	Snyk
CodeQL	1.000	0.000	0.000
Semgrep	0.000	1.000	0.667
Snyk	0.000	0.667	1.000

Table 1.3: IoU for dagger project.

Semgrep and Snyk show a high degree of agreement (IoU = 0.667), indicating that they detect a similar set of vulnerabilities. CodeQL, on the other hand, shows no overlap with either Semgrep or Snyk, suggesting it identifies a completely different set of weaknesses in this project. The venn diagram below visually represents this overlap (Fig. 1.11).

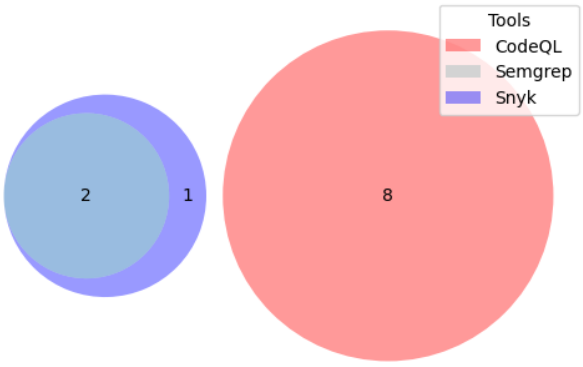

Figure 1.11: Venn diagram for dagger project.

Sentinel project

The IoU values for the sentinel project are as follows:

	CodeQL	Semgrep	Snyk
CodeQL	1.000	0.000	0.057
Semgrep	0.000	1.000	0.200
Snyk	0.057	0.200	1.000

Table 1.4: IoU for sentinel project.

In the case of the sentinel project, all three tools show very low agreement with each other, with the highest IoU being only 0.2 between Semgrep and Snyk. This indicates that the tools are largely complementary for this project, each finding a distinct set of vulnerabilities. The venn diagram below illustrates the limited overlap (Fig. 1.12)..

Guice project

The IoU values for the sentinel project are as follows:

Similar to the sentinel project, the tools show low agreement for the guice project. The IoU between Semgrep and Snyk is 0.2, and there is no overlap between CodeQL and the other two tools. The venn diagram below visualizes this (Fig. 1.13)..

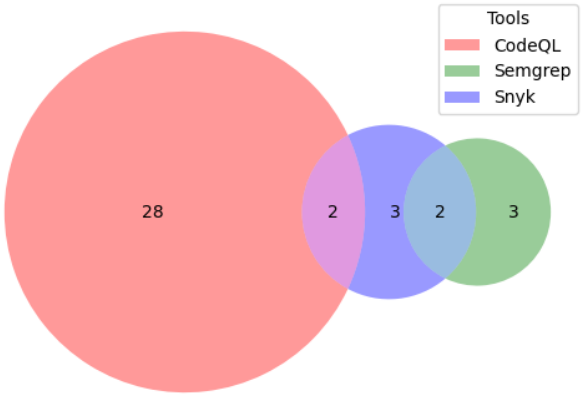

Figure 1.12: Venn diagram for sentinel project.

	CodeQL	Semgrep	Snyk
CodeQL	1.0	0.0	0.0
Semgrep	0.0	1.0	0.2
Snyk	0.0	0.2	1.0

Table 1.5: IoU for guice project.

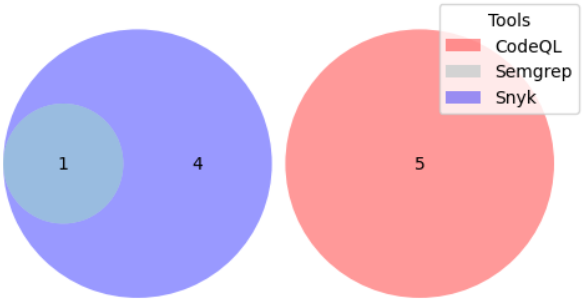

Figure 1.13: Venn diagram for guice project.

1.5 Discussion and Conclusion

This lab provided us with details about the performance of different SAST tools. Our analysis of the CWE Top 25 coverage revealed that Snyk had the broadest coverage among the three tools. However, the IoU analysis showed that no single tool is sufficient for in depth vulnerability detection. The low IoU values across most tool pairs and projects highlight the diversity in their detection capabilities.

To maximize CWE coverage, a combination of tools is necessary. Based on our findings, using CodeQL and Snyk together would provide the best coverage for the analyzed projects, as they tend to find different sets of vulnerabilities. The high IoU between Semgrep and Snyk in some cases suggests that using both might lead to redundant findings.

We understood the importance of a multi-tool approach to static analysis during this lab. Each tool has its own strengths and focuses on different types of weaknesses. By combining the results from multiple tools, we can achieve a more detailed and reliable security assessment. The choice of tools should be governed by the specific programming languages, frameworks, and types of vulnerabilities that are most relevant to the project.

References

[1] MITRE / CWE, "2024 cwe top 25 most dangerous software weaknesses," https://cwe.mitre.org/top25/archive/2024/2024_cwe_top25.html, accessed: 2025-09-25.

[2] W. contributors, "Jaccard index," https://en.wikipedia.org/wiki/Jaccard_index, accessed: 2025-09-25.

[3] K. Li, S. Chen, L. Fan, R. Feng, H. Liu, C. Liu, Y. Liu, and Y. Chen, "Comparison and evaluation on static application security testing (sast) tools for java," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '23)*, 2023, pp. 921–933, accessed: 2025-09-25.

[4] "Lab assignment document," Google Classroom, accessed: 2025-09-27.